

Rings: An Algebraic Structured Peer-to-Peer Network for Decentralized Identity and Resource Ownership

Ryan J. Kung

Abstract

Rings is an implemented structured peer-to-peer network specified by carriers, owner maps, protocol morphisms, and explicit obligations. Its motivating failure mode is authority collapse:

$$\text{root}(id, data, route) \in \{\text{server}, \text{dns}, \text{relay}, \text{platform}\}.$$

Rings replaces that root with self-certifying identity and resource ownership over a CorrectChord overlay, while keeping cryptographic carriers disjoint from the routing carrier. The object studied here is

$$\text{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^\mathcal{E}, \Pi, \mathbb{G}, \mathcal{X})$$

with

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathcal{R}_N &= \text{Correct-Chord overlay object}, \\ \Omega_N(X, H) &= (X, \text{owner}_N \circ H), \\ \tau_\nu &: S_\nu \times I_\nu \rightarrow \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu). \end{aligned}$$

$$\{D, V, M, H, Q, W\} \subset \text{Obj}(\mathbf{Set}/R), \quad \text{CryptoCarrier} \cap R = \emptyset.$$

Claims are stratified as

$$\text{Claim} = \text{Spec} \cup \text{Impl} \cup \text{Evidence}_\Gamma \cup \text{Obligation}.$$

I. Introduction

Centralized platforms are efficient deployment mechanisms but poor authority carriers: identity, naming, storage, and traffic control collapse to service operators [1]–[3]. Rings starts from the opposite requirement:

$$\begin{aligned} \text{Own}(d) &= k_d, \\ \text{Route}(x) &= \text{owner}_N(H_\nu(x)), \\ \text{Run}(\mathcal{P}_\nu) &= \tau_\nu : S_\nu \times I_\nu \rightarrow \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu), \\ \text{RootOfAuthority} &\neq \text{ApplicationServer}. \end{aligned}$$

The central modeling choice is carrier separation. The Chord identifier space is the additive cyclic group for placement and interval order; signatures, threshold schemes, encryption, and zero-knowledge protocols remain in their own fields or groups:

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \text{CC} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}, \\ R \cap \text{CC} &= \emptyset. \end{aligned}$$

This paper studies the algebra that makes the implementation coherent, then marks which claims are implemented, model checked in a finite scope, or still benchmark obligations.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created February 8, 2023; last updated July 4, 2026.

II. Problem Statement and Design Goals

Existing decentralized systems supply only parts of the required object:

$$\begin{aligned} \text{Kademlia} &\Rightarrow \text{lookup}_{XOR} \not\Rightarrow \text{clockwiseOwner}, \\ \text{RelayMixnet} &\Rightarrow \text{pathPrivacy} \not\Rightarrow \text{structuredOwner}, \\ \text{Ledger} &\Rightarrow \text{globalOrder} \not\Rightarrow \text{localOverlay}. \end{aligned}$$

Obligation 1 (Design target).

$$\begin{aligned} G_{id} &= \text{SelfCertID} \wedge \text{DelegatedSession}, \\ G_{own} &= \text{StructuredOwner}, \\ G_{io} &= \text{BrowserNativeTransport}, \\ G_{eff} &= \text{TypedProtocolEffect}, \\ G_{cr} &= \text{CarrierCorrectCrypto}, \\ G_{rep} &= \text{BoundedRepairEvidence}, \\ \text{Rings} &\models G_{id} \wedge G_{own} \wedge G_{io} \wedge G_{eff} \wedge G_{cr} \wedge G_{rep}. \end{aligned}$$

Definition 1 (Methodology).

$$\begin{aligned} \text{Method} &= \text{Mot} \rightarrow \text{Spec}_C \rightarrow \text{Alg}_P \rightarrow \text{ImplMap} \\ &\quad \rightarrow \text{Evidence}_\Gamma \rightarrow \text{Obligation}, \\ \Gamma &= (|N|, \text{topology}, \text{scheduler}, \text{fault}), \\ \text{Admit}(c) &\Rightarrow c \in \text{Spec} \cup \text{Impl} \\ &\quad \cup \text{Evidence}_\Gamma \cup \text{Obligation}. \end{aligned}$$

The prose in this paper is used only where the construction needs motivation, scope, or comparison; the claims themselves are carried by definitions, constraints, invariants, algorithms, and obligations.

$$\text{Contribution} = \{R, \text{Set}/R, \Omega_N, \mathcal{R}_N, \text{RepKey}^\rho, \text{T}_A^\mathcal{E}, \text{Explore}_\Gamma, \text{Report}_{\text{Rings}}\}.$$

III. Algebraic System Model

Assumption 1 (System scope).

$$\begin{aligned} \text{fault} &= \text{fail-stop}, \\ \text{auth}(m, \sigma, k) &= V(k, \sigma, m), \\ \text{net} &= \text{async}, \\ \text{io} &= I_X : A_X^* \rightarrow \text{IO}, \\ \text{consensus} &\notin \text{Overlay}, \\ \text{order} &= \text{SidecarWitness}, \\ \text{crypto_ops} \cap \text{route_ops} &= \emptyset. \end{aligned}$$

The fail-stop clause scopes overlay maintenance; identity, ranking, and application protocols still carry adversarial predicates at their own layer.

Definition 2 (Identifier carrier).

$$\begin{aligned} M &= 2^{160}, \\ R &= \mathbb{Z}/M\mathbb{Z}, \\ \delta_a(x) &= (x - a) \bmod M, \\ (a, b]_R &= \{x \in R : 0 < \delta_a(x) \leq \delta_a(b)\}, \\ \text{RouteOps}_R &= \{0, +, -, \delta_a, (_, _)]_R\}, \\ \text{Mult}_R &\notin \text{RouteOps}_R. \end{aligned}$$

Definition 3 (Live topology).

$$\begin{aligned} N &\subset R, \quad N \neq \emptyset, \\ \text{owner}_N(k) &= \arg \min_{u \in N} \delta_k(u), \\ \text{pred}_N(n) &= \arg \min_{p \in N \setminus \{n\}} \delta_p(n), \\ \text{Succ}_N^q(n) &= \text{take}_q(\text{sort}_{\delta_n}(N \setminus \{n\})), \\ \delta_a|_N &= \text{injective}. \end{aligned}$$

Assumption 2 (Correct-Chord stable base).

$$\begin{aligned} r &= |\text{Succ}_N^r(n)|, \\ \text{Steady}(N) &\Rightarrow |N| \geq r + 1, \\ |N| < r + 1 &\Rightarrow \text{Init}(N), \\ \text{Maintain} &= \{\text{join}, \text{rectify}\} \\ &\quad \cup \{\text{stabilize}\}_{\text{CorrectChord}}. \end{aligned}$$

This is the CorrectChord stable base of Zave [4].

Definition 4 (Affine replica set).

$$\begin{aligned} \rho &\in \mathbb{N}_{>0}, \\ \text{repkey}_i^\rho(k) &= k + \lfloor M \cdot i / \rho \rfloor, \\ \text{RepKey}^\rho(k) &= \langle \text{repkey}_i^\rho(k) : 0 \leq i < \rho \rangle, \\ \text{RepOwner}_N^\rho(k) &= \langle \text{owner}_N(\text{repkey}_i^\rho(k)) : 0 \leq i < \rho \rangle. \\ P_k &= \text{set}(\text{RepKey}^\rho(k)), \\ O_k &= \text{set}(\text{RepOwner}_N^\rho(k)), \\ |P_k| &= \rho, \\ |O_k| &\leq \min(\rho, |N|), \\ \text{collapse}_{N, \rho}(k) &\Leftrightarrow |O_k| < \rho. \end{aligned}$$

Thus replica keys specify redundancy intent; live owner images specify realized redundancy. CorrectChord successor lists remain the topology safety root.

Definition 5 (Rings overlay object).

$$\begin{aligned} \mathcal{R}_N &= (R, N, \text{owner}_N, \text{pred}_N, \text{Succ}_N^r, \text{RepOwner}_N^\rho, F, E), \\ F &\subseteq N \times \mathbb{N} \times N, \\ E &= \prod_{v \in \text{VID}} E_v, \\ \text{SafetyRoot}(\mathcal{R}_N) &= (\text{pred}_N, \text{Succ}_N^r), \\ \text{PerfWitness}(\mathcal{R}_N) &= F. \end{aligned}$$

Proposition 1 (Rotation equivariance).

$$\begin{aligned} t \in R, \quad N + t &= \{n + t : n \in N\} \Rightarrow \\ \text{owner}_{N+t}(k + t) &= \text{owner}_N(k) + t. \end{aligned}$$

Proof.

$$\begin{aligned} \forall n \in N. \quad \delta_{k+t}(n + t) &= ((n + t) - (k + t)) \bmod M \\ &= \delta_k(n). \end{aligned}$$

□

Constraint 1 (Carrier separation).

$$\begin{aligned} \text{PlacementOps} &= \{0, +, -, \delta_a, (a, b]_R\} \\ &\quad \cup \{\text{owner}_N\} \\ &\quad \cup \{\text{RepKey}^\rho, \text{RepOwner}_N^\rho\}, \\ \text{CryptoOps} &= \{\times, ^{-1}, \cdot_{\text{scalar}}, \text{interp}\} \\ &\quad \cup \{\text{pair}, \text{subgroup}\}, \\ \text{PlacementOps} \cap \text{CryptoOps} &= \emptyset, \\ \text{CryptoCarrier} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}. \end{aligned}$$

Object	Carrier or law
Chord IDs	additive cyclic group R
DID proof	signature verifier and transcript law
VID	$H_\nu : X_\nu \rightarrow R$ then owner map
placement	join semilattice or explicit finalizer
Storage	group operation plus byte-to-point
merge	adapter
E2E encryption	partial order over witness domains
Sequencing	writer monad over action lists
Effects	

Table 1

Carrier boundaries in the Rings model

IV. Architecture As Interfaces

Definition 6 (Layer tuple).

$$\mathcal{A} = (I, T, O, X, P, L)$$

$$\begin{array}{l|l} I & (DID, \Pi, \text{Session}) \\ T & (C, A_T, \text{offer}, \text{answer}, \text{send}, \text{recv}) \\ O & \mathcal{R}_N \\ X & (S_X, A_X, \text{cap}, I_X) \\ P & \{\mathcal{P}_\nu\}_{\nu \in \text{Namespace}} \\ L & \text{App} \circ P \end{array}$$

Definition 7 (Identity proof).

$$\Pi = (K, \Sigma, V, D, \text{enc}, \text{ttl})$$

$$\begin{aligned} V &: K \times \Sigma \times B^* \rightarrow \{\top, \perp\}, \\ D &: K \rightarrow R, \\ \text{enc} &: \Sigma \rightarrow B^*, \\ \text{ttl} &: \Sigma \rightarrow \text{Time}. \end{aligned}$$

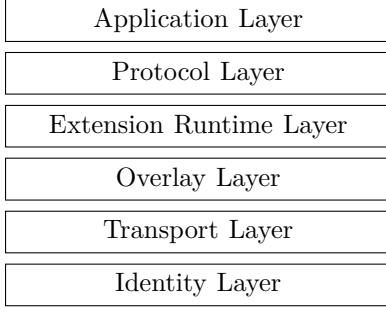


Figure 1. Layers of Rings Network

Invariant 1 (Session authority).

$$d \in X_D, \quad s \in K.$$

$$\text{Session}(d, s, e) \Rightarrow V(k_d, \sigma_{d \rightarrow s}, d \| s \| e) = \top \wedge \text{now} < e.$$

$$\text{HopCheck}(d, s, e) \equiv \text{Session}(d, s, e).$$

Definition 8 (Transport interface).

$$T = (C, \text{offer}, \text{answer}, \text{send}, \text{recv})$$

$$A_T = \{\text{Offer}, \text{Answer}, \text{Ice}, \text{Open}, \text{Send}, \text{Recv}, \text{Close}\}.$$

$$I_T : A_T^* \rightarrow \text{IO}_{WebRTC} \cup \text{IO}_{Native}.$$

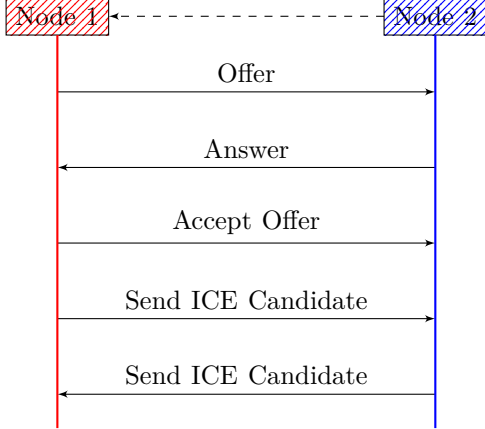


Figure 2. Standard SDP/ICE exchange

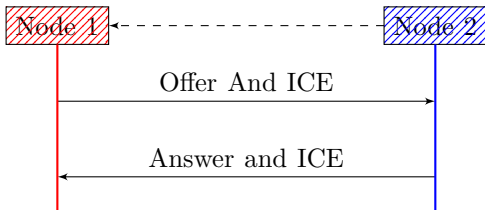


Figure 3. Rings single round-trip SDP/ICE exchange

Definition 9 (Runtime interpreter).

$$\mathcal{X} = (S_X, A_X, \text{cap}, I_X)$$

$$\begin{aligned} \text{cap} &\subseteq A_X, \\ I_X &: A_X^* \rightarrow \text{IO}, \\ I_X(\epsilon) &= \text{Noop}, \\ I_X(\alpha \cdot \beta) &= I_X(\alpha); I_X(\beta). \end{aligned}$$

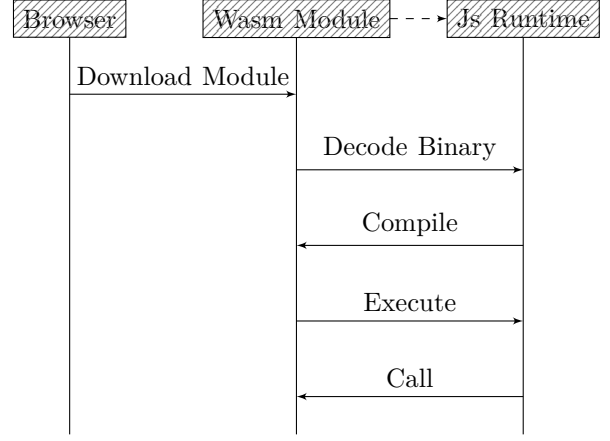


Figure 4. Wasm module execution boundary

Obligation 2 (Runtime confinement).

$$\begin{aligned} \tau &: S \times I \rightarrow \mathcal{E} + (S \times O \times A_X^*), \\ \text{emit}(\tau(s, i)) &\subseteq \text{cap}, \\ a \notin \text{cap} &\Rightarrow I_X(a) = \text{Reject}(a). \end{aligned}$$

V. Category and Effects

Definition 10 (Namespace object).

$$\begin{aligned} \nu &\in \{D, V, M, H, U, Q, W\}, \\ H_\nu &: X_\nu \rightarrow R, \\ (X_\nu, H_\nu) &\in \mathbf{Set}/R. \end{aligned}$$

Definition 11 (Placement functor).

$$\begin{aligned} \Omega_N &: \mathbf{Set}/R \rightarrow \mathbf{Set}/N, \\ (X, H) &\mapsto (X, \text{owner}_N \circ H). \end{aligned}$$

$$P_\nu^N = \text{owner}_N \circ H_\nu : X_\nu \rightarrow N.$$

Proposition 2 (Placement functoriality).

$$\begin{aligned} f : X_\nu \rightarrow X_\mu, \quad H_\mu \circ f &= H_\nu. \\ P_\mu^N \circ f &= P_\nu^N. \end{aligned}$$

Proof.

$$\text{owner}_N \circ H_\mu \circ f = \text{owner}_N \circ H_\nu.$$

□

Definition 12 (Writer effect monad).

$$\begin{aligned} A^* &= \mathbf{FreeMonoid}(A), \\ \mathsf{T}_A(X) &= X \times A^*, \\ \eta(x) &= (x, \epsilon), \\ (x, \alpha) \gg= f &= \text{let } f(x) = (y, \beta) \text{ in } (y, \alpha \cdot \beta). \end{aligned}$$

Proposition 3 (Effect associativity).

$$(h^* \circ g^*) \circ f^* = h^* \circ (g^* \circ f^*).$$

Proof.

$$((\alpha \cdot \beta) \cdot \gamma) = (\alpha \cdot (\beta \cdot \gamma)).$$

Definition 13 (Typed failure stack).

$$\begin{aligned} \mathsf{T}_A^\mathcal{E}(X) &= \mathcal{E} + (X \times A^*). \\ \pi_A(e \in \mathcal{E}) &= \epsilon, \\ \pi_A(x, \alpha) &= \alpha, \\ \text{RecoverableReport} &\in A. \end{aligned}$$

Definition 14 (Protocol object).

$$\begin{aligned} \mathcal{P}_\nu &= (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu) \\ \tau_\nu : S_\nu \times I_\nu &\rightarrow \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu). \end{aligned}$$

Definition 15 (Protocol morphism).

$$\begin{aligned} \Sigma : \mathcal{P}_\nu &\rightarrow \mathcal{P}_\mu = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*) \\ \sigma_A^* &: A_\nu^* \rightarrow A_\mu^*, \\ \sigma_A^*(\alpha \cdot \beta) &= \sigma_A^*(\alpha) \cdot \sigma_A^*(\beta), \\ H_\mu \circ \sigma_X &= H_\nu, \\ \mathsf{T}_{\sigma_A^*}^{\sigma_\mathcal{E}}(\sigma_S \times \sigma_O) \circ \tau_\nu &= \tau_\mu \circ (\sigma_S \times \sigma_I). \end{aligned}$$

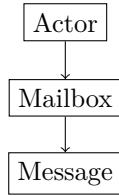


Figure 5. Actor Model

VI. Chord Overlay

$$\begin{aligned} \text{Base} &= \text{Chord}, \\ \text{Maintenance} &= \text{CorrectChord}, \\ \text{Object} &= \mathcal{R}_N, \\ \text{Witnesses} &= \{\text{Algorithms 1, 2, 3}\}. \end{aligned}$$

This section fixes Chord as the base protocol and CorrectChord as the maintenance law [4], [5].

Definition 16 (Local node state).

$$\begin{aligned} S_n &= (p_n, L_n, F_n, E_n) \\ p_n &\in N \cup \{\perp\}, \\ L_n &\in N^{\leq r}, \\ F_n &: \mathbb{N} \rightarrow N, \\ E_n &\subseteq E, \\ p_n = \text{pred}_N(n), \quad L_n &= \text{Succ}_N^r(n). \end{aligned}$$

Definition 17 (Finger target).

$$\begin{aligned} \text{target}_i(n) &= n + 2^i, \\ \text{finger}_i(n) &= \text{owner}_N(\text{target}_i(n)), \\ F_n(i) &\in \{\perp, \text{finger}_i(n)\}. \end{aligned}$$

Invariant 2 (Ring fixpoint).

$$\begin{aligned} \text{Inv}_{\text{ring}}(N, S) &\equiv \forall n \in N. p_n = \text{pred}_N(n) \\ &\quad \wedge L_n = \text{Succ}_N^r(n). \end{aligned}$$

Algorithm 1 DHT Join Transitions

```

1: function BeginJoin( $n, \text{known}$ )
2:    $\text{pred}[n] \leftarrow \text{none}$ 
3:    $a \leftarrow \text{FindSuccessor}(n.\text{did})$ 
4:   return  $\text{Remote}(\text{known}, a)$ 
5: function InstallSuccessor( $n, s$ )
6:    $\text{succ}[n] \leftarrow \text{take}(r, [s])$ 
7:   return  $\text{Remote}(s, \text{QuerySuccList}(s))$ 
8: function InstallSuccList( $n, s, \text{list}$ )
9:    $\text{succ}[n] \leftarrow \text{take}(r, s :: \text{list})$ 
10:  if  $\text{head}(\text{succ}[n]) \neq \text{none}$  then
11:     $h \leftarrow \text{head}(\text{succ}[n])$ 
12:     $\text{notify} \leftarrow \text{Notify}(h, n.\text{did})$ 
13:     $\text{sync} \leftarrow \text{SyncStorage}(h)$ 
14:    return  $\text{Multi}(\text{notify}, \text{sync})$ 
15:  else
16:    return  $\text{Noop}$ 

```

Algorithm 2 DHT Lookup Algorithm

```

1: function Lookup( $\text{key}, \text{node}$ )
2:    $s \leftarrow \text{head}(\text{succ}[\text{node}])$ 
3:   if  $s = \text{none}$  then
4:     return  $\text{Repair}(\text{QuerySuccList}(\text{node}))$ 
5:   if  $\text{key} \in (\text{node}.\text{did}, s]$  then
6:     return  $\text{Local}(s)$ 
7:    $\text{next} \leftarrow \text{closest\_preceding\_node}(\text{node}, \text{key})$ 
8:   if  $\text{next} = \text{none}$  then
9:     return  $\text{Repair}(\text{QuerySuccList}(s))$ 
10:  else
11:     $a \leftarrow \text{FindSuccessor}(\text{key})$ 
12:    return  $\text{Remote}(\text{next}, a)$ 

```

Invariant 3 (Lookup owner preservation).

$$\begin{aligned} &\text{Steady}(N) \wedge \text{Inv}_{\text{ring}}(N, S) \\ &\Rightarrow \text{Lookup}(S, k) \Downarrow \text{owner}_N(k). \\ &\Downarrow = \text{Reach}_{A_D}(2, 3). \end{aligned}$$

VII. DID and Resource Algebra

Definition 18 (DID namespace).

$$\begin{aligned} \mathcal{D} &= (X_D, H_D, \Pi, \text{Session}), \quad H_D : X_D \rightarrow R. \\ D(k) &= H_D(k), \\ \text{owner}_N(D(k)) &\in N, \\ \text{Session} &\subseteq X_D \times K \times \text{Time}. \end{aligned}$$

Definition 19 (VID namespace).

$$\begin{aligned} \mathcal{V} &= (X_V, H_V, \text{Auth}_V), \\ H_V &: X_V \rightarrow R, \\ \text{PrivateKey}(H_V(x)) &= \perp, \\ \text{Auth}_V(\text{op}, x, \sigma, \text{caps}) &\in \{\top, \perp\}. \end{aligned}$$

Definition 20 (Biased order).

$$A \leq_C B \iff \delta_C(A) \leq \delta_C(B).$$

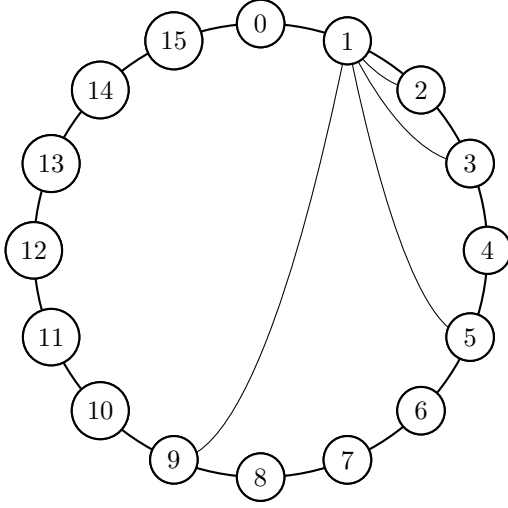


Figure 6. Chord finger targets from node 1 in a 16-node ring

Algorithm 3 DHT Stabilization

```

1: procedure Stabilize( $n, topo$ )
2:    $actions \leftarrow []$ 
3:    $c \leftarrow topo.pred$ 
4:    $s \leftarrow head(succ[n])$ 
5:   if  $c \neq none$  and  $c \in (n.did, s)$  then
6:      $succ[n] \leftarrow insert(succ[n], c)$ 
7:      $actions \leftarrow actions :: FindSuccessor(c)$ 
8:    $succ[n] \leftarrow take(r, merge(succ[n], topo.succ))$ 
9:   if  $head(succ[n]) \neq none$  then
10:     $h \leftarrow head(succ[n])$ 
11:     $actions \leftarrow actions :: Notify(h, n.did)$ 
12:   return  $Multi(actions)$ 

```

Definition 21 (Replicated entry algebra).

$$\begin{aligned} \mathcal{E}_v^{join} &= (E_v, \sqsubseteq, \sqcup, \perp), \\ \mathcal{E}_v^{final} &= (E_v, \text{conflict}_v, \text{finalize}_v). \end{aligned}$$

$$\begin{aligned} x \sqcup y &= y \sqcup x, \\ (x \sqcup y) \sqcup z &= x \sqcup (y \sqcup z), \\ x \sqcup x &= x, \\ x \sqcup \perp &= x, \\ \text{finalize}_v &: \text{conflict}_v(E_v) \rightarrow E_v. \end{aligned}$$

case₁ = CRDT.

Algorithm 4 Biased Circular Comparison

```

1: function BiasedCompare( $A, B, C$ )
2:    $a \leftarrow \delta_C(A)$ 
3:    $b \leftarrow \delta_C(B)$ 
4:   if  $a < b$  then
5:     return  $A <_C B$ 
6:   else if  $a > b$  then
7:     return  $B <_C A$ 
8:   else
9:     return  $A = B$ 

```

This is the CRDT join-semilattice case [6].

Invariant 4 (Placement).

$$\begin{aligned} K_{\nu,x} &= \text{replicaKeys}(H_\nu(x)), \\ P_{\nu,x} &= \text{set}(\text{RepKey}^\rho(H_\nu(x))), \\ \text{Inv}_{\text{place}} &\equiv \forall \nu, x. K_{\nu,x} \subseteq P_{\nu,x}. \end{aligned}$$

Obligation 3 (Replica realization).

$$\begin{aligned} O_{\nu,x} &= \text{set}(\text{RepOwner}_N^\rho(H_\nu(x))), \\ L_{\nu,x} &= |O_{\nu,x}|, \\ \text{FullRed}_{\nu,x} &\Leftrightarrow L_{\nu,x} = \min(\rho, |N|), \\ \text{Collapse}_{\nu,x} &\Leftrightarrow L_{\nu,x} < \min(\rho, |N|), \\ \text{LiveClaim}_{\nu,x}(\rho) &\Rightarrow \text{FullRed}_{\nu,x}, \\ \text{Report}_{\text{Rings}} &\ni |\{(\nu, x) : \text{Collapse}_{\nu,x}\}|. \end{aligned}$$

Definition 22 (Mailbox).

$$\begin{aligned} mbox(d, e) &= H_M(\text{rings.mailbox} \| d \| e), \\ entry_M &= (\eta, exp, proof_s, dst, cipher). \end{aligned}$$

Definition 23 (Hidden service descriptor).

$$\begin{aligned} h &= (\nu, \rho, Intro, Relay, \\ &\quad Cap, K, exp, rank), \\ VID(h) &= H_H(h), \\ \text{Lookup}(h) &= P_H^N(h), \\ \text{Valid}(h, \sigma) &= \text{Auth}_H(h, \sigma). \end{aligned}$$

Definition 24 (Subring).

$$\begin{aligned} \mathcal{S}_u &= (u, N_u, \text{owner}_{N_u}, \text{Succ}_{N_u}^{r_u}, E_u) \\ u &= H_U(policy), \\ memberlog_u &\in \text{MergeLog} \cup \text{FinalState}. \end{aligned}$$

VIII. Cryptographic Carriers and Traffic

Definition 25 (End-to-end carrier).

$$\begin{aligned} |\mathbb{G}| &= q, \\ \text{Scalar}(\mathbb{G}) &= \mathbb{Z}_q, \\ x &\in \mathbb{Z}_q, \\ h &= g^x, \\ \mathbb{G} &\neq R. \end{aligned}$$

Algorithm 5 ElGamal Encryption and Decryption

```

1: Input: Encoded group element  $m_G \in \mathbb{G}$ , private key
    $x \in \mathbb{Z}_q$ , public key  $h = g^x$ , generator  $g$ 
2: Output: Group ciphertext  $(c_1, c_2)$ 
3: procedure Encryption
4:   Choose a random integer  $k \in \mathbb{Z}_q$ 
5:   Compute  $c_1 = g^k$ 
6:   Compute  $c_2 = m_G \cdot h^k$ 
7:   return  $(c_1, c_2)$ 
8: procedure Decryption
9:   Compute  $m_G = c_2 \cdot (c_1)^{-x}$ 
10:  return  $m_G$ 

```

Algorithm 5 is the group-level carrier operation. Byte payloads must first be encoded into curve points and split into bounded frames. The direct ElGamal body is malleable when detached from its signed transport

envelope; Rings therefore treats message signatures, DID ownership of the public key, stream identifiers, sequence numbers, and final-frame checks as part of the security boundary rather than as optional metadata.

Definition 26 (Signed encrypted frame).

$$f = (sid, seq, last, cipher, pk_s, \sigma_s).$$

$$\begin{aligned} m_f &= sid || seq || last || cipher, \\ \text{FrameValid}_\Delta(f) &\Leftrightarrow V(pk_s, \sigma_s, m_f) = \top \\ &\quad \wedge \text{Session}(D(pk_s), sid, e) \\ &\quad \wedge \text{SeqOK}_\Delta(seq, last). \end{aligned}$$

Definition 27 (Receiver sequence policy).

$$\Delta = (next, final, closed, pending, seen, w).$$

$$\begin{aligned} \text{NoFuture}_\Delta(seq) &\Leftrightarrow \neg \exists p \in pending. p > seq, \\ \text{Dup}_\Delta(seq) &\Leftrightarrow seq \in seen \cup pending, \\ \text{New}_\Delta(seq, last) &\Leftrightarrow seq \notin seen \cup pending \\ &\quad \wedge \neg closed \\ &\quad \wedge seq - next \leq w \\ &\quad \wedge (final = \perp \vee seq \leq final) \\ &\quad \wedge (last \Rightarrow \text{NoFuture}_\Delta(seq)), \\ \text{SeqOK}_\Delta(seq, last) &\Leftrightarrow \text{Dup}_\Delta(seq) \\ &\quad \vee \text{New}_\Delta(seq, last), \\ \text{Dup}_\Delta(seq) &\Rightarrow \text{emit} = \epsilon. \end{aligned}$$

Here w is receiver policy, not a signed wire field.

Definition 28 (Secret sharing placement).

$$\begin{aligned} P &\in \mathbb{F}_q[X], \\ sh_i &= (i, P(i)) \in \mathbb{F}_q^2, \\ H_S(sh_i) &\in R, \\ \text{reconstruct}(\{sh_i\}) &= P(0) \quad (\mathbb{F}_q\text{-interp.}) \end{aligned}$$

This is Shamir reconstruction over the separate field carrier [7].

Definition 29 (Relay frame).

$$r = (path, dst, v, session, payload, report).$$

$$\begin{aligned} \text{RelayValid}(r) &= \text{SessionValid}(session) \\ &\quad \wedge dst \in R, \end{aligned}$$

$$\text{SplitVocabulary} = \text{MSRP}.$$

The split vocabulary follows MSRP [8].

Algorithm 6 End-to-End Chunking Algorithm

```

1: procedure E2E Chunking
2:   Input: data, chunk size
3:   Output: chunks
4:   chunks  $\leftarrow \square$ 
5:   total-chunks  $\leftarrow \text{ceil}(\text{len}(\text{data})/\text{chunk-size})$ 
6:   for i in range(total-chunks) do
7:     chunk  $\leftarrow \text{data}[i*\text{chunk-size} : (i+1)*\text{chunk-size}]$ 
8:     append chunk to chunks
9:   return chunks

```

Invariant 5 (Chunk reassembly).

$$\begin{aligned} \text{complete}(m) &\Leftrightarrow \forall i < \text{total}(m). \exists! c_i \\ &\quad \wedge \sum_i |c_i| \leq B_m \\ &\quad \wedge \text{now} < \text{ttl}(m). \end{aligned}$$

IX. Ordering, Ranking, and Security

Definition 30 (Sidecar witness).

$$w = (\text{domain}, \text{root}, \text{height}, \text{time}, \text{finality}).$$

$$c = H(\text{msg}),$$

$$\begin{aligned} \text{Witness}(\text{msg}, w) &\Leftrightarrow \text{Include}(c, w.\text{root}) = \top \\ &\quad \wedge \text{Final}(w) = \top. \end{aligned}$$

Definition 31 (Witness order).

$$\begin{aligned} a < b &\Leftrightarrow \text{domain}(w_a) = \text{domain}(w_b) \\ &\quad \wedge \text{time}(w_a) < \text{time}(w_b). \end{aligned}$$

$$\chi : \text{Domain}_a \times \text{Domain}_b \rightarrow \text{Order}.$$

Definition 32 (Ranking event).

$$\begin{aligned} q &= (\text{subject}, \text{rater}, \text{behavior}, \\ &\quad \text{evidence}, \text{epoch}, \text{weight}, \sigma). \end{aligned}$$

$$m_q = \text{subject} || \text{behavior} || \text{evidence} || \text{epoch},$$

$$\text{RankValid}(q) \Leftrightarrow V(k_{\text{rater}}, \sigma, m_q) = \top.$$

Definition 33 (Score aggregation).

$$\begin{aligned} \text{score}_e(s) &= \text{robust}\{\text{weight}(q) \cdot \text{value}(q) : \\ &\quad q.\text{subject} = s\}. \end{aligned}$$

$$\text{robust} \in \{\text{median}, \text{trimmedMean}, \text{declared}\}.$$

Attack	Predicate	Required boundary
Sybil	many DIDs, one actor	admission and ranking policy
Replay	stale valid σ	TTL, nonce cache, stream/final
MITM	wrong key for DID	DID proof and signed envelope
DoS	unbounded resource use	quotas and chunk bounds

Table II

Security predicates and required controls

$$\begin{aligned} \text{Base} &\not\models \text{SybilResist} \wedge \text{Anon} \wedge \text{DoSResist}, \\ \text{SecClaim} &\Rightarrow \text{Adm} \wedge \text{Rank} \wedge \text{Relay} \wedge \text{Quota}. \end{aligned}$$

Obligation 4 (Replay rejection).

$$\begin{aligned} \text{accept}(s, \eta, t) &\Rightarrow \eta \notin \text{Seen}_s \wedge t < \text{exp}(s), \\ \text{Seen}'_s &= \text{Seen}_s \cup \{\eta\}, \\ \text{emitDelivery} &< \text{Seen}'_s. \end{aligned}$$

X. Verification Evidence and Evaluation Scope

Spec $\neq$ Evidence_R $\neq$ Evaluation.

The repository contains executable evidence for finite overlay, storage, and message-flow obligations; it does not yet contain a wide-area Rings benchmark.

Definition 34 (State relation).

$$\frac{C = (S, Q) \quad Q = a :: Q_0 \quad I_A(a) = i \quad \tau(S, i) = (S', O, \beta)}{(S, Q) \rightarrow (S', Q_0 \cdot \beta)}$$

Explore_R = ModelCheck($\rightarrow, \Gamma$)

$\Gamma = (|N|, \text{topology}, \text{scheduler}, \text{fault}),$
ModelClaim(Γ) $\Rightarrow$ finite(Γ) $\wedge$ declared(Γ).

The relation follows TLA+ style and Stateright execution in the declared finite scope [9], [10].

Invariant 6 (Effect refinement).

$$\text{Impl}_A(s, i) \preceq I_A^\#(\tau(s, i)).$$

$$\pi_{\text{state}}(\text{Impl}_A) = \pi_{\text{state}}(I_A^\# \circ \tau).$$

Obligation 5 (Topology coverage).

$$\mathcal{L} = \{\text{line}, \text{wrap}, \text{gap}, \text{partition}, \text{merge}, \text{churn}\}$$

$$\mathcal{S} = \{\text{loss}, \text{dup}, \text{reorder}, \text{timeout}, \text{restart}\}.$$

$$\text{CheckCase} = (L, S, P), \quad L \in \mathcal{L}, \quad S \in \mathcal{S}.$$

Artifact	Checked scope	Limitation
Topology tests	join, rectify, stabilize fixpoints on finite DID layouts	not a parametric TLAPS proof
Stateright	predecessor update, small discovery abstraction, storage CRDT, handoff	scoped models, not a full live network
Trace replay	deterministic delivery schedules over real routing code	not an Internet latency benchmark
E2E tests	signed direct-ElGamal frame order, final marker, and replay behavior	not a symbolic cryptographic proof

Table III

Verification evidence and explicit limits

Definition 35 (Latency decomposition).

$$T_{\text{total}} = T_{\text{conn}} + h \cdot T_{\text{hop}} + T_{\text{crypto}} + T_{\text{app}} + T_{\text{witness}}.$$

$$h = O(\log |N|) \quad (F_n \text{ populated}).$$

Definition 36 (Chord performance baseline).

$$\begin{aligned} \text{fail}(Q, F) &= \frac{|F|}{|Q|}, \\ \text{timeout}(Q) &= \frac{1}{|Q|} \sum_{q \in Q} \text{to}(q), \\ \eta &= (|N|, \text{churn}, \text{stabilize}, \text{loss}), \\ B_C(\eta) &= (\mathbb{E}[h_C], \text{timeout}(Q_C), \text{fail}(Q_C, F_C)), \\ \mathbb{E}[h_C] &\in O(\log |N|). \end{aligned}$$

Obligation 6 (Benchmark report).

$$\text{Report}_{\text{Rings}} = (N, \rho, r, \text{transport}, \text{runtime}, \text{scheduler}, \vec{T}_R, \text{timeout}(Q_R), \text{fail}(Q_R, F_R)).$$

$$B_C(\eta) \neq \text{Report}_{\text{Rings}}.$$

$$\begin{aligned} \vec{T}_R &= (T_{\text{conn}}, h_R T_{\text{hop}}, T_{\text{crypto}}, T_{\text{app}}, T_{\text{witness}}), \\ \text{Overhead}_{R/C} &= (T_{\text{conn}}, T_{\text{crypto}}, T_{\text{app}}, T_{\text{witness}}, \text{runtime}), \\ \text{PerfClaim}_R &\Rightarrow \text{Measure}_{\text{build}}(\text{Report}_{\text{Rings}}) \wedge \text{Cite}(B_C). \end{aligned}$$

Chord supplies the lookup/churn baseline [5]; Rings performance requires Rings-specific measurements.

XI. Related Work

Use the comparison coordinates

$$\text{Axis}(S) = (\mu, \omega, \chi, \lambda).$$

$$\begin{aligned} \mu &= \text{metric}, \\ \omega &= \text{owner}, \\ \chi &= \text{order}, \\ \lambda &= \text{runtime}. \end{aligned}$$

S	Axis(S)
Kad	(XOR, bucket, $\perp$, native)
Chord	(circ, succ, $\perp$, native)
Tor/Nym	(path, $\perp$, $\perp$, relay)
Ledger	($\perp$, acct, global, repl)
Rings	(circ, owner _N , sidecar, web $\cup$ native)

Kademlia/IPFS/libp2p supply scalable lookup but not the clockwise owner law needed here [11]–[13]. Tor and Nym supply relay-path privacy, not structured ownership [14], [15]. Ledgers supply global order; Rings keeps ordering outside the base overlay. WebRTC and WebAssembly are runtime adapters, so the same protocol object is interpreted in browser and native environments [16], [17].

System	Metric	Ownership law
Kademlia	XOR	bucket routing
Relay mixnets	path	no structured owner
Rings	circular	clockwise owner, successor list, affine placement keys

Table IV

Routing and ownership comparison [11]–[15]

XII. Conclusion

The paper's claim is the following structured object, not an informal bundle of network mechanisms:

$$\text{Rings} = (R, \mathcal{R}_N, \text{Set}/R, \Omega_N, \mathbb{T}_A^\varepsilon, \Pi, \mathbb{G}, \mathcal{X}).$$

Overlay	=	$\text{CorrectChord}(R, N, r),$
Resource	=	$(X_\nu, H_\nu) \in \mathbf{Set}/R,$
Placement	=	$\Omega_N(X_\nu, H_\nu),$
Transition	=	$\tau_\nu : S_\nu \times I_\nu \rightarrow Y_\nu,$
Y_ν	=	$\mathbf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu),$
Crypto	=	$\mathbb{G} \vee \mathbb{F}_q, \quad \text{Crypto} \cap R = \emptyset,$
Evidence	=	$\{\text{scopedModels}, \text{traceReplay}\}$ $\cup \{\text{invariants}, \text{benchmarkObligations}\}.$

References

- [1] N. Kshetri, “Privacy and security issues in cloud computing: The role of institutions and institutional evolution,” *Telecommunications Policy*, vol. 37, no. 4–5, pp. 372–386, 2013.
- [2] D. Zissis and D. Lekkas, “Addressing cloud computing security issues,” *Future Generation Computer Systems*, vol. 28, no. 3, pp. 583–592, 2012.
- [3] N. Kshetri and S. Murugesan, “Cloud computing and EU data privacy regulations,” *Computer*, vol. 46, no. 3, pp. 86–89, 2013.
- [4] P. Zave, “Reasoning about identifier spaces: How to make Chord correct,” *IEEE Transactions on Software Engineering*, vol. 43, no. 12, pp. 1144–1156, 2017, <https://arxiv.org/abs/1610.01140>.
- [5] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, “Chord: A scalable peer-to-peer lookup service for internet applications,” in *Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*. ACM, 2001, pp. 149–160.
- [6] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Stabilization, Safety, and Security of Distributed Systems*, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.
- [7] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [8] B. Campbell, R. Mahy, and C. Jennings, “The message session relay protocol (MSRP),” RFC 4975, 2007, <https://www.rfc-editor.org/info/rfc4975>.
- [9] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [10] J. Nadal, “Stateright: Model checking for implementing distributed systems,” <https://www.stateright.rs/>, accessed: July 4, 2026.
- [11] P. Maymounkov and D. Mazières, “Kademlia: A peer-to-peer information system based on the XOR metric,” in *Peer-to-Peer Systems*, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 53–65.
- [12] J. Benet, “IPFS - content addressed, versioned, P2P file system,” arXiv:1407.3561 [cs.NI], 2014, <https://arxiv.org/abs/1407.3561>.
- [13] Protocol Labs, “libp2p: A modular network stack,” 2026, <https://libp2p.io/>.
- [14] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The Second-Generation onion router,” in *13th USENIX Security Symposium (USENIX Security 04)*. San Diego, CA: USENIX Association, Aug. 2004, <https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router>.
- [15] C. Diaz, H. Halpin, and A. Kiayias, “The Nym network: The next generation of privacy infrastructure,” Whitepaper, 2021, <https://nym.com/nym-whitepaper.pdf>.
- [16] H. Alvestrand, “Overview: Real-time protocols for browser-based applications,” RFC 8825, 2021, <https://www.rfc-editor.org/rfc/rfc8825>.
- [17] WebAssembly Working Group, “WebAssembly core specification,” W3C Recommendation, 2019, <https://www.w3.org/TR/2019/REC-wasm-core-1-20191205/>.